

Reviewer Pack — Minimal Rank of Tropicalized Sheaves vs ACC⁰ Circuit Lower B...

Entry 0ef1ef61998e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 08:25:34 UTC

Minimal Rank of Tropicalized Sheaves vs ACC⁰ Circuit Lower Bounds for Explicit Functions

Entry ID: 0ef1ef61998e **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 08:25:34 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Tropical Geometry) **Field B** (complexity object): Complexity Theory (ACC⁰ Circuit Lower Bounds)

Statement:

[‘For an explicit function f in P with circuit size n , let F be its associated tropical variety. If the minimal rank of the sheaf on F is greater than k , then f requires an ACC⁰ circuit of size at least $2^k + n$.’, ‘Equivalently, for a fixed constant c , if there exists an explicit function f such that $\text{MinRank}(\text{Sheaf}(F)) > k$ and $\text{CircuitSize}(f) \leq 2^{(k-c)n}$, then the sheaf on F is trivial.’, ‘For any given instance of an explicit function in P , determine its minimal rank sheaf and circuit size to verify the above conditions.’]

Rationale (proposer’s reasoning):

[‘Tropical geometry provides a powerful tool for studying properties of functions over posets, which can be applied to understand the structure of circuits.’, ‘Sheaves in algebraic geometry capture local information about varieties, which may reveal hidden structures that are relevant to circuit complexity.’, ‘The conjecture suggests that by examining tropicalized sheaves, we could obtain insights into ACC⁰ circuit lower bounds for explicit functions.’]

Taxonomy category: ACC_SIPSER (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 6f9f02ffe1b53896

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all explicit functions in P with known circuit size n , the ratio of minimal rank of the sheaf on the tropical variety to the ACC⁰ circuit lower bound is greater than $2^k / (2^{(k-c)} + 1)$, and no function violates this condition. The conjecture is falsified if there exists a function in P with $\text{MinRank}(\text{Sheaf}(F)) > k$ and $\text{CircuitSize}(f) \leq 2^{(k-c)n}$, such that the ratio does not hold.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Rank Tropicalized Sheaves AND ACC^o Circuit Lower Bounds - Sheaf on Tropical Variety AND ACC^o Circuit Size - Tropical Geometry MinRank Sheaf OR ACC^o Circuit Complexity

Top relevant hits considered: - [<http://arxiv.org/abs/1807.06283v2>] Tropical Fano Schemes - [<http://arxiv.org/abs/0811.1949v2>] Moduli Spaces of Semistable Sheaves on Projective Deligne-Mumford Stacks - [<http://arxiv.org/abs/2311.01864v1>] SortNet: Learning To Rank By a Neural-Based Sorting Algorithm - [<http://arxiv.org/abs/1207.2443v2>] Tropical Teichmuller and Siegel spaces - [<http://arxiv.org/abs/1204.3875v2>] Tropicalizing vs Compactifying the Torelli morphism - [<http://arxiv.org/abs/1204.6154v2>] Local Tropicalization - [<http://arxiv.org/abs/1207.1925v1>] Introduction to tropical algebraic geometry - [<http://arxiv.org/abs/1206.1925v1>] Counting Algebraic Curves with Tropical Geometry

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A, b):
        n = len(b)
        for i in range(n):
            max_row = i + max(range(i, n), key=lambda k: abs(A[k][i]))
            A[i], A[max_row] = A[max_row], A[i]
            b[i], b[max_row] = b[max_row], b[i]
            factor = A[i][i]
            for j in range(n):
                A[i][j] /= factor
            b[i] /= factor
            for k in range(n):
                if k != i:
                    factor = A[k][i]
                    for j in range(n):
```

```

        A[k][j] -= factor * A[i][j]
        b[k] -= factor * b[i]
    return b

def min_rank(A):
    n = len(A)
    rank = 0
    for i in range(n):
        if all(abs(A[j][i]) < 1e-9 for j in range(rank)):
            continue
        rank += 1
        for j in range(i, n):
            A[i], A[j] = A[j], A[i]
            b[i], b[j] = b[j], b[i]
    return rank

def circuit_size(f):
    # Placeholder function to simulate circuit size calculation
    return len(f)

def tropical_variety(f):
    # Placeholder function to simulate tropical variety computation
    n = len(f)
    A = [[0 for _ in range(n)] for _ in range(n)]
    b = [0 for _ in range(n)]
    for i in range(n):
        for j in range(i+1, n):
            A[i][j] = abs(f[i] - f[j])
            b[i] += A[i][j]
    return gaussian_elimination(A, b)

def min_rank_sheaf(F):
    # Placeholder function to simulate minimal rank of sheaf computation
    return len(F) - 1

n = random.randint(5, 40)
f = [random.random() for _ in range(n)]
F = tropical_variety(f)
sheaf_rank = min_rank_sheaf(F)
circ_size = circuit_size(f)

metric_value = sheaf_rank / circ_size
conjecture_holds = metric_value > 2**k / (2**(k - c) + 1)
counterexample = "" if conjecture_holds else f"Function with n={n}, sheaf_rank={sheaf_rank}, c={c}

return {
    "metric_name": "Ratio of MinRank(Sheaf(F)) to CircuitSize(f)",
    "metric_value": metric_value,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 30)) + [37, 41, 43,

```

```

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"Function with n={results[0]['instances_tested']}\"")
else:
    print("RESULT: INCONCLUSIVE insufficient support for conjecture")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a386d864.py", line 94, in <module>

result = run_trial(seed)

^^^^^^^^^^^^^^^^

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a386d864.py", line 72, in run_trial

F = tropical_variety(f)

^^^^^^^^^^^^^^^^

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a386d864.py", line 64, in tropical_variety

return gaussian_elimination(A, b)

^^^^^^^^^^^^^^^^

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a386d864.py", line 29, in gaussian_elimination

A[i][j] /= factor

ZeroDivisionError: division by zero

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing any data, which prevents us from verifying the conjecture's conditions. | next: Re-run the test without errors to verify the conjecture's conditions.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14683
2	preregistration	ollama_remote	glm4:latest	0	0	7371
3	novelty	ollama_remote	glm4:latest	0	0	4891
4	novelty	ollama_remote	glm4:latest	0	0	5919
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	40744
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7715
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7717
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11832
9	judge	ollama_remote	glm4:latest	0	0	7996

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 108867 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/0ef1ef61998e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0ef1ef61998e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/0ef1ef61998e.tar.gz` (if generated)